Московский авиационный институт
(национальный исследовательский университет)

Кафедра 806 «Вычислительная математика и программирование»

Объектно-ориентированное программирование (семестр 1)

01.03.02 «Прикладная математика и информатика»

ЛАБОРАТОРНАЯ РАБОТА 3: МНОЖЕСТВО ОБЪЕКТОВ С ДИНАМИЧЕСКИМ МАССИВОМ И ПЕРЕГРУЗКОЙ ОПЕРАТОРОВ

Московский авиационный институт
(национальный исследовательский университет)

Кафедра 806 «Вычислительная математика и программирование»

Объектно-ориентированное программирование (семестр 1)

01.03.02 «Прикладная математика и информатика»

Общая информация

Срок сдачи	2 недели
Предварительные требования	Лекции 1–7, лабораторные работы 1–2
Стандарт языка	C++20
Сборка	CMake 3.20+
Тестирование	Google Test
Среда	macOS / Linux / Windows, компилятор с поддержкой C++20
Вариант	1–20, по номеру в журнале

Цели работы

1. Реализовать **класс-контейнер** с ручным управлением динамической памятью.
2. Применить **правило пяти**: копирование, перемещение, присваивание и деструктор.
3. Перегрузить операторы так, чтобы их семантика была **очевидной и осмысленной**.
4. Реализовать теоретико-множественные операции: объединение, пересечение, разность.
5. Поддержать **инварианты элементов**: канонизацию, нормализацию, валидацию.
6. Покрыть класс модульными тестами Google Test и проверить отсутствие утечек.

Задание

Каждый студент реализует **конкретный класс множества** (без шаблонов), который хранит динамический массив объектов заданного типа. Элемент множества — небольшой класс с operator==, operator<<, а для sorted-вариантов ещё и operator<.

Множество хранит **не более одного экземпляра** каждого элемента. Порядок хранения зависит от варианта:

- **unsorted** — элементы хранятся в порядке добавления;
- **sorted** — элементы хранятся в порядке, заданном operator<;
- **канонизированное** — элементы приводятся к каноническому виду перед сравнением (дроби, температуры, телефоны, интервалы).

Скелет класса, правило пяти и базовые операции одинаковы для всех вариантов. Меняются тип элемента, стратегия поиска/вставки/удаления, рост capacity и набор дополнительных операторов.

Ограничения

Запрещено:

- шаблоны (template);

- контейнеры STL (std::vector, std::map, std::set, std::list, …);
- умные указатели (std::unique_ptr, std::shared_ptr, std::weak_ptr);
- std::array, std::span, std::optional.

Разрешено:

- new / delete, new[] / delete[];
- <iostream>, <cstring>, <cmath>, <cassert>;
- <cstddef> для std::size_t;
- <utility> для std::move и std::swap;
- собственные вспомогательные функции и классы.

Минимальный контракт класса

Для варианта с элементом X класс называется XSet (например, PointSet, FractionSet, StudentSet) и обязан предоставлять:

```
class XSet {
public:
    XSet();
    XSet(const XSet&);
    XSet(XSet&&) noexcept;
    XSet& operator=(const XSet&);
    XSet& operator=(XSet&&) noexcept;
    ~XSet();

    bool add(const X&);              // true — множество изменилось
    bool remove(const X&);           // true — элемент был и удалён
    bool contains(const X&) const;
    bool operator()(const X&) const; // принадлежность

    std::size_t size() const;        // количество элементов
    bool empty() const;              // size() == 0

    XSet& operator+=(const X&);
    XSet& operator-=(const X&);

    XSet operator|(const XSet&) const; // объединение
    XSet operator&(const XSet&) const; // пересечение
    XSet operator-(const XSet&) const; // разность

    bool operator==(const XSet&) const;
    bool operator!=(const XSet&) const;

    friend std::ostream& operator<<(std::ostream&, const XSet&);
};
```

Пояснения к контракту

- **add** возвращает true, если множество изменилось: элемент добавлен. Если элемент уже есть, возвращается false. Для варианта 19 (upsert) и варианта 10 (слияние интервалов) семантика уточнена в описании варианта: add возвращает true, если новое значение заменило старое или множество изменилось после слияния.
- **remove** возвращает true, только если элемент действительно был удалён.
- **contains** и **operator()** не меняют множество и работают за время, определяемое вариантом (линейный поиск или бинарный).
- **operator+= / operator-=** добавляют/удаляют один элемент и возвращают ссылку на *this для цепочки вызовов.
- **|, &, -** возвращают **новое** множество и не меняют операнды.
- == сравнивает множества как множества: порядок хранения не важен (для sorted-вариантов порядок канонический, поэтому можно сравнивать последовательности).
- **operator<<** печатает элементы в фигурных скобках через запятую: { (0, 0), (1, 2) }. Для sorted-вариантов — в отсортированном порядке, для unsorted — в порядке хранения.

Требования к элементу множества

Элемент X обязан предоставлять:

1. **Конструктор по умолчанию** — контейнер выделяет массив new X[capacity].
2. **Копирующий конструктор и копирующее присваивание** — глубокие, если элемент владеет динамической памятью (Student, Book).
3. **bool operator==(const X&) const** — равенство с учётом канонизации.
4. **std::ostream& operator<<(std::ostream&, const X&)** — вывод в поток.
5. **bool operator<(const X&) const** — только для sorted-вариантов.
6. **Правило пяти** — если элемент владеет памятью, он тоже обязан его соблюдать.
7. **Валидация/канонизация** — если вариант этого требует (даты, время, IP, ISBN, дроби, температуры, телефоны).

Элемент **не должен** использовать std::string. Для строковых полей (Student::name, Book::title) используйте char* и strlen/strcpy/strcmp из <cstring>, либо фиксированный массив char.

Управление памятью и правило пяти

Внутреннее представление множества:

```
X* data_;            // динамический массив элементов
std::size_t size_;   // сколько элементов хранится
std::size_t capacity_; // сколько элементов помещается без роста
```

Обязательные требования:

1. **Деструктор** освобождает data_ через delete[].
2. **Копирующий конструктор** выделяет новый массив и копирует элементы (глубокая копия, а не копирование указателя).
3. **Копирующее присваивание** реализуется через **copy-and-swap**: сильная гарантия исключений, корректная работа с самоприсваиванием.
4. **Перемещающий конструктор** забирает data_, обнуляет источник и помечен noexcept.
5. **Перемещающее присваивание** освобождает свой ресурс, забирает чужой, корректно обрабатывает самоперемещение и помечено noexcept.
6. **Рост capacity** — по стратегии варианта:
   - точный +1 (вариант 5) — показать дороговизну;
   - +4 (вариант 15) — показать фрагментацию;
   - удвоение (варианты 9 и др.) — амортизированно эффективно;
   - capacity == 0 → первый рост до 1 (или до 4).
7. **Удаление** — по стратегии варианта: сдвиг с сохранением порядка или swap-with-last без сохранения порядка.

Пример copy-and-swap:

```
XSet& XSet::operator=(const XSet& other) {
    if (this != &other) {
        XSet tmp(other);        // может бросить — исходный объект не изменён
        swap(tmp);              // noexcept
    }
    return *this;
}
```

```
void XSet::swap(XSet& other) noexcept {
    std::swap(data_, other.data_);
    std::swap(size_, other.size_);
    std::swap(capacity_, other.capacity_);
}
```

Операции над множествами

Семантика операций:

- A | B — все элементы, которые есть в A **или** в B;
- A & B — элементы, которые есть **и** в A, **и** в B;
- A - B — элементы A, которых **нет** в B;
- A ^ B — элементы, которые есть **ровно в одном** из множеств (симметрическая разность; в вариантах, где она требуется);
- A <= B — A является подмножеством B (в вариантах, где требуется).

Тестирование

Тесты обязательны. Минимальный набор для любого варианта:

1. пустое множество: size() == 0, contains ложен;
2. add нового элемента — true, дубликата — false;
3. remove существующего — true, отсутствующего — false;
4. contains и operator() согласованы;
5. копия независима: изменение копии не меняет оригинал;
6. перемещение оставляет источник в корректном пустом состоянии;
7. |, &, - дают ожидаемые множества;
8. == и != согласованы;
9. вывод operator<< соответствует формату;
10. stress-тест: 1000 добавлений, проверка размера и отсутствия дубликатов.

Дополнительно проверьте сборку с санитайзерами (пример):

```
cmake -S . -B build-asan -DCMAKE_CXX_FLAGS="-fsanitize=address,undefined -g"
cmake --build build-asan
./build-asan/lab_03_tests
```

Варианты индивидуальных заданий

№	Тип элемента	Алгоритмическая особенность
1	Point2D — точка (x, y)	Базовый unsorted-вариант. Динамический массив без порядка; add ищет дубликат линейно и дописывает в конец; remove сдвигает элементы; объединение, пересечение, разность через вложенные циклы.
2	Complex — комплексное число	Sorted + binary search. Массив отсортирован по (re, im); add вставляет в нужную позицию; contains — бинарный поиск; объединение, пересечение, разность — двухпутевое слияние.
3	Fraction — дробь	Канонизация элемента. Перед сравнением дробь нормализуется: знак, сокращение через НОД. Множество unsorted; remove через swap-with-last, порядок не сохраняется; объединение через повторный add.
4	Date — дата	Валидация + sorted insert. add проверяет корректность даты и вставляет в отсортированный массив; remove — бинарный поиск + сдвиг; == через слияние.
5	Time — время (h, m, s)	Простая стратегия роста capacity +1. Показать дороговизну точного realloc; unsorted; ^ реализуется как разность объединения и пересечения.
6	Money — сумма + валюта	Sorted + subset. Сортировка по сумме и валюте; объединение и пересечение через слияние; дополнительно operator<= — проверка подмножества через слияние; присваивание через copy-and-swap.
7	PhoneNumber — телефон	Нормализация перед сравнением. Хранит цифры; add сравнивает нормализованные номера; unsorted; remove swap-with-last; разность через вложенные циклы.

№	Тип элемента	Алгоритмическая особенность
8	ColorRGB — цвет	Sorted lexicographically + symmetric difference. Бинарный поиск; operator^ через двухпутевое слияние; вывод всегда в отсортированном виде.
9	Vector3D — 3D-вектор	Unsorted + capacity doubling. add дописывает; remove сдвигает; == через size() и contains; & через вложенные циклы; рост capacity удвоением.
10	Interval — отрезок [l, r]	Инвариант: множество непересекающихся отрезков. add при пересечении/касании объединяет отрезки; объединение и пересечение учитывают канонизацию; sorted по левой границе.
11	Student — имя char* + id	Элемент сам владеет динамической памятью. Нужно правило 5 и для Student, и для StudentSet; сравнение по id; unsorted; copy-and-swap в присваивании.
12	Book — название char* + год	Sorted по названию через strcmp. Бинарный поиск; объединение, пересечение, разность через слияние; operator[] только для чтения с проверкой границ.
13	Temperature — значение + единица	Канонизация к одной единице. Перед сравнением температура приводится к канонической единице; unsorted; add ищет дубликат после канонизации; remove сдвигом.
14	GeoCoordinate — широта/долгота	Sorted + subset. Сортировка по широте, затем долготе; бинарный поиск; operator<= — подмножество через слияние; operator() — принадлежность.
15	Matrix2x2 — фиксированная матрица 2×2	Unsorted + рост capacity на +4. Элемент простой, но сравнение покомпонентное; remove swap-with-last; объединение через вложенные циклы; показать фрагментацию при маленьком шаге

№	Тип элемента	Алгоритмическая особенность
		роста.
16	Quadratic — квадратный трёхчлен (a, b, c)	Sorted lexicographically + разность слиянием. add вставляет по (a, b, c); - через двухпутевое слияние; вывод в каноническом порядке.
17	PlayingCard — карта (масть, ранг)	Sorted + простая модель. Сортировка по масти и рангу; бинарный поиск; объединение, пересечение слиянием; operator< для карт; хорош для тестирования границ.
18	IPAddress — 4 октета	Валидация + sorted numeric. add проверяет октеты 0..255 и вставляет по числовому порядку; remove бинарным поиском; operator[] read-only.
19	ExamGrade — id студента + оценка	Upsert-семантика. Множество хранит не более одной оценки на студента: add при совпадении id заменяет оценку, а не отбрасывает; объединение и пересечение работают по id; полезно для объяснения разницы между set и map-подобным поведением.
20	ISBN — 13 цифр	Валидация контрольной суммы + sorted. add проверяет checksum и вставляет в отсортированный массив; remove бинарным поиском; ^ через двухпутевое слияние.

Что именно варьируется

- **Представление:** unsorted-массив, sorted-массив, множество с канонизацией (интервалы, дроби).
- **Поиск:** линейный или бинарный.
- **Вставка:** в конец, в отсортированную позицию, с заменой существующего элемента.
- **Удаление:** сдвиг с сохранением порядка или swap-with-last без сохранения.
- **Рост capacity:** точный +1, +4, удвоение.
- **Операции множеств:** вложенные циклы или двухпутевое слияние.

- **Дополнительные операторы:** ^, <= (подмножество), [], ().
- **Элемент с динамической памятью:** Student, Book — добавляют правило пяти на уровне элемента.
- **Инвариант элемента:** Fraction, Temperature, Interval, ISBN.

Рекомендации

1. **Сначала элемент, потом множество.** Реализуйте и протестируйте X (==, <, <<, канонизацию), затем переходите к XSet.
2. **Правило пяти — до операций.** Пока копирование и удаление не работают, операции над множествами тестировать бессмысленно.
3. **new X[capacity] требует конструктора по умолчанию.** Если элемент его не имеет, добавьте X() = default или явный конструктор по умолчанию.
4. **Проверяйте capacity == 0.** Первый add должен корректно выделить память.
5. **remove с swap-with-last** меняет порядок — это допустимо только в вариантах, где порядок не важен. В sorted-вариантах удаляйте сдвигом.
6. **Бинарный поиск требует operator<**, а не только ==. Убедитесь, что сортировка и поиск используют одно и то же отношение порядка.
7. **Канонизация должна быть идемпотентной:** повторная нормализация не меняет объект.
8. **Для char*** не забудьте strlen + 1 под '\0', глубокую копию и delete[] в деструкторе.
9. **Тестируйте границы:** пустое множество, один элемент, дубликаты, удаление последнего, самоприсваивание, перемещение из пустого.